COLECCIÓN BOSÓN

APIS
Y ARQUITECTURAS WEB MODERNAS

REST · SOAP · GraphQL · HTTP · Swagger

Diseño, Documentación y Buenas Prácticas



Arquitectura Cliente — API — Servidor

Autor **Ing. Milton Cayo Blanco**

Colección Bosón
Área APIs y Arquitecturas Web
Versión 1.0

A mis estudiantes, técnicos e ingenieros,
que cada día buscan dominar el arte
de conectar sistemas, servicios y personas
a través de interfaces bien diseñadas.

A quienes entienden que una buena API
no es solo código, sino un contrato
entre el que construye y el que confía.

— Ing. Milton Cayo Blanco

Físico e Ingeniero

Ingeniero Electromecánico — UMSA
Ingeniero Informático — Ontario, Canadá
Mecánica — NASA (National Aeronautics and Space Administration)
CEO Gravity — Fundación Aeroespacial en Bolivia
Físico — UMSA

Prólogo

Las interfaces de programación de aplicaciones —conocidas universalmente como **APIs**— constituyen hoy el tejido invisible que sostiene la economía digital. Cada vez que un usuario solicita un Uber, consulta el clima, realiza un pago o recibe una recomendación de contenido, decenas de APIs trabajan en milisegundos para orquestar la respuesta. Entender cómo funcionan, cómo se diseñan bien y cómo se documentan correctamente es, en nuestros tiempos, una competencia fundamental para cualquier ingeniero de software.

Este libro nace de la necesidad de dotar a los profesionales de herramientas conceptuales y prácticas para construir APIs robustas, seguras y mantenibles. He comprobado que muchos proyectos fracasan no por falta de código, sino por falta de **diseño**: endpoints inconsistentes, verbos HTTP mal utilizados, documentación inexistente o contratos implícitos que nadie entiende después de seis meses. Este texto ataca esos problemas de raíz.

El viaje comienza con los **fundamentos de las APIs**: qué son, qué tipos existen (REST, SOAP, GraphQL), cómo funciona el protocolo HTTP y cuál es la diferencia entre cliente y servidor. Luego avanzamos hacia el **diseño orientado a recursos**: principios REST, naming conventions, versionado, idempotencia y relaciones entre entidades. Finalmente, abordamos la **documentación profesional** con Swagger y OpenAPI, herramienta que permite generar documentación automática y realizar pruebas interactivas en tiempo real.

La **Colección Bosón** mantiene su compromiso de ofrecer material técnico riguroso y accesible. Este Tomo 1 sienta las bases para volúmenes posteriores que explorarán autenticación OAuth 2.0, APIs en tiempo real con WebSockets y despliegue de microservicios.

Ing. Milton Cayo Blanco
Autor, Colección Bosón

Lista de Acrónimos

API	Application Programming Interface — Interfaz de Programación de Aplicaciones
REST	Representational State Transfer
SOAP	Simple Object Access Protocol
GraphQL	Graph Query Language — Lenguaje de Consulta de Grafos
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
JSON	JavaScript Object Notation
XML	Extensible Markup Language
URL	Uniform Resource Locator
URI	Uniform Resource Identifier
CRUD	Create, Read, Update, Delete
EDA	Event-Driven Architecture — Arquitectura Orientada a Eventos
OAS	OpenAPI Specification
JWT	JSON Web Token

Índice general

Prólogo iii

Lista de Acrónimos v

1 Fundamentos de APIs 1

1.1 Introducción a las APIs . . . 1

1.1.1 ¿Qué es una API? . . . 1

1.1.2 Tipos de APIs . . . 2

1.1.3 Cliente vs Servidor . . . 4

1.1.4 Arquitectura Web . . . 5

1.2 Protocolo HTTP . . . 5

1.2.1 Métodos HTTP . . . 5

1.2.2 Códigos de Estado HTTP . . . 6

1.2.3 Headers y Body . . . 6

1.2.4 JSON vs XML . . . 7

1.3 Ejercicios Propuestos . . . 8

2 Diseño de APIs 11

2.1 Principios de Diseño REST . . . 11

2.1.1 Recursos y Endpoints . . . 11

2.1.2 Naming Conventions . . . 11

2.1.3 Versionado de APIs . . . 12

2.1.4 Buenas Prácticas REST . . . 12

2.2 Diseño Orientado a Recursos . . . 13

2.2.1 CRUD y Operaciones Estándar . . . 13

2.2.2 Relaciones entre Entidades . . . 13

2.2.3 Idempotencia . . . 13

2.3 Documentación . . . 14

2.3.1 Uso de Swagger / OpenAPI . . . 14

2.3.2 Generación Automática de Documentación . . . 15

2.3.3 Testing Interactivo . . . 16

2.4 Ejercicios Resueltos — Implementación paso a paso . . . 17

2.5 Ejercicios Propuestos . . . 22

Referencias 27

Configuración del Entorno de Desarrollo 29

.1 Instalación de FastAPI y Uvicorn . . . 29

.2 Herramientas de Testing de APIs . . . 29

Recursos y Referencias en Línea 31

Capítulo 1

Fundamentos de APIs

Las APIs son el lenguaje universal con el que los sistemas informáticos se comunican entre sí. Desde una aplicación móvil que consulta datos meteorológicos hasta un sistema bancario que procesa transacciones en tiempo real, detrás de cada operación existe una interfaz bien definida que hace posible el intercambio de información. Este capítulo construye los fundamentos conceptuales y técnicos que todo profesional debe dominar antes de diseñar o consumir cualquier API.

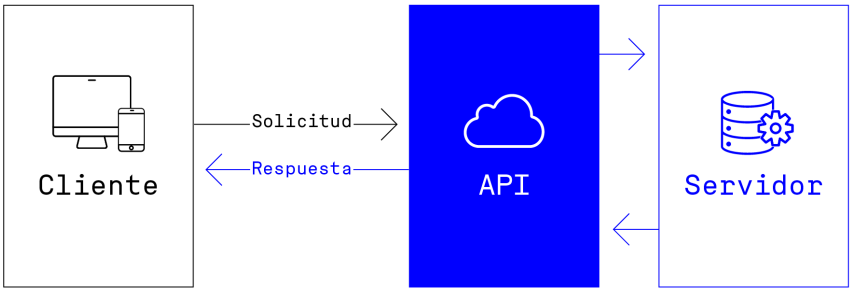


Figura 1.1. Api - Application Programming Interface

1.1 Introducción a las APIs

1.1.1 ¿Qué es una API?

Una **API** (Application Programming Interface) es un conjunto de definiciones y protocolos que permite que diferentes aplicaciones de software se comuniquen entre sí. Funciona como un *contrato*: especifica qué operaciones están disponibles, qué datos se deben enviar y qué respuesta se puede esperar.

Nota Técnica

Analogía práctica: Una API es como el menú de un restaurante. El cliente (tu aplicación) no necesita saber cómo se prepara la comida (la lógica interna del servidor). Solo necesita conocer el menú (la documentación de la API), hacer su pedido (la *request*) y recibir su plato (la *response*).

Una API expone **endpoints**, que son puntos de acceso específicos a través de los cuales se realizan las operaciones. Por ejemplo:

```
GET  https://api.ejemplo.com/v1/usuarios          # Listar todos
    los usuarios
```



Figura 1.2. Analogía - Menú de un Restaurante - Cliente

```
GET  https://api.ejemplo.com/v1/usuarios/42      # Obtener
   usuario con ID 42
POST https://api.ejemplo.com/v1/usuarios         # Crear un nuevo
   usuario
```

Código 1.1. Ejemplos de endpoints de una API de usuarios

1.1.2 Tipos de APIs

Existen tres grandes arquitecturas para el diseño de APIs. Cada una tiene sus fortalezas y casos de uso específicos:



Figura 1.3. Tipos de Api aplicativas y de servicio

REST (Representational State Transfer)

REST es el estilo arquitectónico dominante en el desarrollo web moderno. Se basa en seis principios fundamentales: interfaz uniforme, cliente-servidor, sin estado (*stateless*), caché, sistema en capas y código bajo demanda (opcional).

- Utiliza el protocolo HTTP de forma semántica (GET, POST, PUT, DELETE).
- Los recursos se identifican mediante URLs.
- Las respuestas suelen ser en formato JSON.
- Es ligero, escalable y fácil de consumir desde cualquier plataforma.

SOAP (Simple Object Access Protocol)

SOAP es un protocolo de mensajería basado en XML. Es más estricto y formal que REST, lo que lo hace preferido en entornos empresariales con requisitos estrictos de seguridad y transacciones.

- Define un contrato explícito mediante WSDL (*Web Services Description Language*).
- Utiliza exclusivamente XML como formato de intercambio.
- Soporta transacciones ACID y protocolos de seguridad WS-Security.
- Más verboso y complejo que REST, pero más robusto para sistemas críticos.

```xml
<soap:Envelope
   xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Header/>
  <soap:Body>
    <obtenerUsuario>
      <id>42</id>
    </obtenerUsuario>
  </soap:Body>
</soap:Envelope>
```

Código 1.2. Ejemplo de mensaje SOAP (envelope)

GraphQL

GraphQL es un lenguaje de consulta para APIs desarrollado por Facebook en 2015. A diferencia de REST, el cliente define exactamente qué datos necesita, eliminando el problema de *over-fetching* (recibir más datos de los necesarios) y *under-fetching* (tener que realizar múltiples peticiones).

```graphql
query {
  usuario(id: 42) {
    nombre
    email
    pedidos {
      total
      estado
    }
  }
}
```

Código 1.3. Consulta GraphQL: solo los campos necesarios

Parámetros / Operandos

Comparación rápida:

Criterio	REST	SOAP	GraphQL
Formato	JSON / XML	XML	JSON
Protocolo	HTTP	HTTP / SMTP	HTTP
Curva de aprendizaje	Baja	Alta	Media
Flexibilidad	Media	Baja	Alta
Uso ideal	Web / Móvil	Empresa	Frontend complejo

1.1.3 Cliente vs Servidor

En cualquier comunicación mediante APIs existe una distinción fundamental entre dos roles: el **cliente** y el **servidor**.

El **cliente** es el sistema que realiza solicitudes. Puede ser una aplicación web, una app móvil, un servicio backend o incluso un script de automatización. El cliente no conoce (ni necesita conocer) los detalles internos de implementación del servidor.

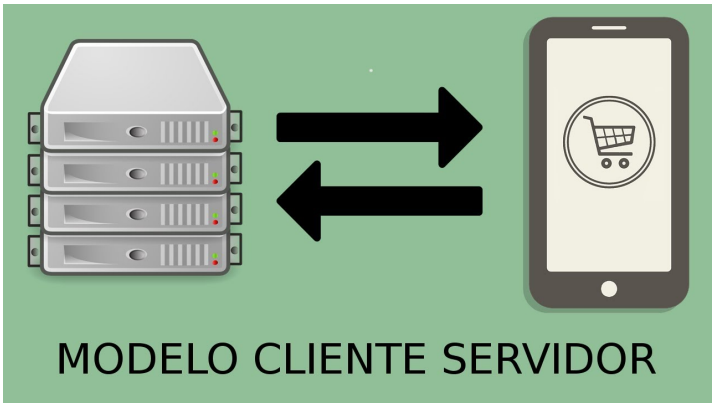


Figura 1.4. Cliente Servidor

El **servidor** es el sistema que recibe las solicitudes, las procesa y devuelve una respuesta. Contiene la lógica de negocio, accede a las bases de datos y aplica las reglas de seguridad.

Ejemplo Analítico — Paso a Paso

Flujo completo de una petición REST:

1. El cliente construye la petición HTTP con método, URL, headers y body.
2. La petición viaja por la red hasta el servidor.
3. El servidor autentica, valida y procesa la solicitud.
4. El servidor consulta la base de datos o servicio correspondiente.
5. El servidor construye la respuesta con código de estado y payload JSON.
6. La respuesta regresa al cliente, que la interpreta y renderiza.

1.1.4 Arquitectura Web

Las APIs modernas operan dentro de una arquitectura web multicapa. Comprender esta arquitectura es esencial para diseñar APIs escalables y seguras:

- **Capa de presentación:** Aplicaciones web o móviles que consumen la API.
- **API Gateway:** Punto de entrada centralizado que gestiona autenticación, rate limiting, logging y enrutamiento hacia los microservicios.
- **Capa de servicios:** Microservicios o monolitos que implementan la lógica de negocio y exponen los endpoints.
- **Capa de datos:** Bases de datos relacionales, NoSQL, cachés y servicios de almacenamiento que persisten la información.

1.2 Protocolo HTTP

1.2.1 Métodos HTTP

El protocolo **HTTP** (Hypertext Transfer Protocol) define los métodos que indican la acción que el cliente desea realizar sobre un recurso. En el diseño REST, cada método tiene una semántica precisa que debe respetarse estrictamente:

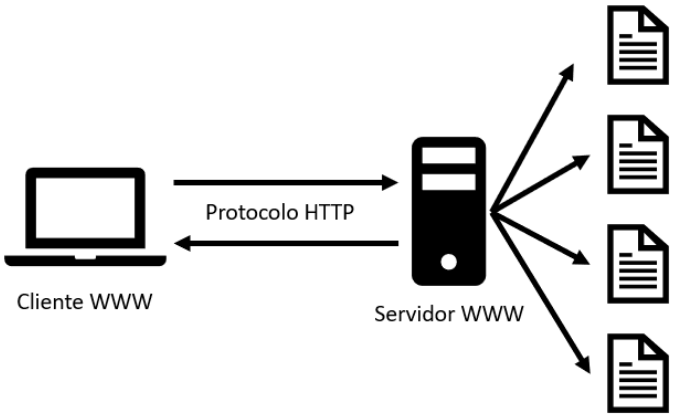


Figura 1.5. Protocolo HTTP

Parámetros / Operandos

Método	Semántica	Descripción
`GET`	Lectura	Obtiene un recurso. Nunca debe modificar datos.
`POST`	Creación	Crea un nuevo recurso. No idempotente.
`PUT`	Reemplazo	Reemplaza un recurso completo. Idempotente.
`PATCH`	Modificación parcial	Actualiza campos específicos del recurso.
`DELETE`	Eliminación	Elimina el recurso identificado por la URL.

Advertencia

Error frecuente: Usar `GET` para operaciones que modifican datos (por ejemplo, `GET /usuarios/borrar/42`) viola el principio de idempotencia y puede causar borrados accidentales al ser una URL almacenada en caché o compartida. Siempre usa el método semánticamente correcto.

1.2.2 Códigos de Estado HTTP

Los **códigos de estado** son números de tres dígitos que el servidor incluye en cada respuesta para indicar el resultado de la operación. Se agrupan en cinco categorías:

- **1xx — Informativos:** La solicitud fue recibida y está siendo procesada.
- **2xx — Éxito:** La solicitud fue recibida, entendida y procesada correctamente.
- **3xx — Redirección:** Se requiere una acción adicional para completar la solicitud.
- **4xx — Error del cliente:** La solicitud contiene errores o no puede ser procesada.
- **5xx — Error del servidor:** El servidor falló al procesar una solicitud válida.

Códigos más utilizados en APIs REST:

`200 OK` — Éxito general (GET, PUT, PATCH).
`201 Created` — Recurso creado exitosamente (POST).
`204 No Content` — Operación exitosa sin cuerpo de respuesta (DELETE).
`400 Bad Request` — Datos de entrada inválidos o malformados.
`401 Unauthorized` — No autenticado; se requieren credenciales.
`403 Forbidden` — Autenticado pero sin permisos suficientes.
`404 Not Found` — El recurso solicitado no existe.
`409 Conflict` — Conflicto de estado (ej: registro duplicado).
`500 Internal Server Error` — Error inesperado en el servidor.

1.2.3 Headers y Body

Una petición HTTP se compone de tres partes principales: la **línea de solicitud**, los **headers** y el **body**.

```
POST /v1/usuarios HTTP/1.1
Host: api.ejemplo.com
Content-Type: application/json
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
Accept: application/json

{
  "nombre": "Ana Quispe",
  "email": "ana@ejemplo.com",
  "rol": "editor"
}
```

Código 1.4. Anatomía de una petición HTTP POST

Los **headers** más relevantes en el diseño de APIs son:

- `Content-Type`: Indica el formato del cuerpo enviado (`application/json`).
- `Accept`: Indica el formato esperado en la respuesta.
- `Authorization`: Transmite credenciales de autenticación (Bearer token, API key).
- `Cache-Control`: Define políticas de almacenamiento en caché.
- `X-Request-ID`: Identificador único de la petición para trazabilidad.

1.2.4 JSON vs XML

XML

```
<empinfo>
   <employees>
      <employee>
         <name>James Kirk</name>
         <age>40></age>
      </employee>
      <employee>
         <name>Jean-Luc Picard</name>
         <age>45</age>
      </employee>
      <employee>
         <name>Wesley Crusher</name>
         <age>27</age>
      </employee>
   </employees>
</empinfo>
```

JSON

```
{  "empinfo" :
     {
          "employees" :  [
          {
               "name" : "James Kirk",
               "age" : 40,
          },
          {
               "name" : "Jean-Luc Picard",
               "age" : 45,
          },
          {
               "name" : "Wesley Crusher",
               "age" : 27,
          }
                         ]
     }
}
```

Figura 1.6. Json - XML

Los dos formatos predominantes para el intercambio de datos en APIs son **JSON** y **XML**:

```
// JSON (recomendado para REST)
{
  "id": 42,
  "nombre": "Ana Quispe",
  "activo": true,
  "roles": ["editor", "revisor"]
}

<!-- XML (usado en SOAP y sistemas legacy) -->
<usuario>
  <id>42</id>
  <nombre>Ana Quispe</nombre>
  <activo>true</activo>
  <roles>
    <rol>editor</rol>
    <rol>revisor</rol>
  </roles>
```

```
</usuario>
```

Código 1.5. El mismo recurso en JSON y XML

Nota Técnica

JSON es el estándar de facto para REST. Es más ligero, legible y fácil de parsear en todos los lenguajes modernos. XML sigue siendo relevante en integraciones con sistemas empresariales legacy (SAP, BizTalk) y en SOAP. Para APIs públicas nuevas, siempre prefiere JSON.

1.3 Ejercicios Propuestos

Ejercicio 1

Dado el endpoint `/api/productos/15`, indique qué método HTTP utilizaría para:

- Obtener el producto
- Actualizar completamente el producto
- Eliminar el producto

Justifique cada elección.

Solución:

- Obtener el producto: **GET**, porque se utiliza para consultar datos sin modificarlos.
- Actualizar completamente el producto: **PUT**, ya que reemplaza todo el recurso.
- Eliminar el producto: **DELETE**, porque elimina el recurso identificado.

Ejercicio 2

Diseñe una API REST para gestionar libros en una biblioteca digital. Incluya al menos:

- Endpoint para listar libros
- Endpoint para crear un libro
- Endpoint para obtener un libro por ID

Solución:

- Listar libros: `GET /api/libros`
- Crear libro: `POST /api/libros`
- Obtener libro por ID: `GET /api/libros/{id}`

Ejercicio 3

Explique el flujo completo de una petición desde un cliente móvil hasta un servidor cuando se realiza un login.

Solución:

1. El usuario ingresa credenciales en la app móvil

2. La app envía una petición HTTP POST al servidor
3. La petición pasa por internet hacia el servidor
4. El backend valida credenciales en la base de datos
5. Si son correctas, genera un token (ej. JWT)
6. El servidor responde con el token
7. El cliente almacena el token para futuras peticiones

Ejercicio 4

Compare REST y GraphQL en términos de:

- Flexibilidad
- Cantidad de datos transferidos
- Complejidad de implementación

Solución:

- Flexibilidad: GraphQL es más flexible porque permite pedir solo los datos necesarios.
- Cantidad de datos: REST puede enviar datos innecesarios; GraphQL optimiza la respuesta.
- Complejidad: REST es más simple; GraphQL es más complejo de implementar.

Ejercicio 5

Analice el siguiente endpoint:

```
POST https://api.tienda.com/v1/ordenes
```

Indique:

- Qué acción realiza
- Qué tipo de datos esperaría en el body

Solución:

- Acción: Crear una nueva orden en el sistema.
- Datos esperados: Información del cliente, lista de productos, cantidades, dirección de envío y método de pago.

Ejercicio 6

Diseñe endpoints para una API donde un usuario puede tener múltiples pedidos.
Solución:

- Obtener pedidos de un usuario: `GET /api/usuarios/{id}/pedidos`
- Crear pedido: `POST /api/usuarios/{id}/pedidos`
- Obtener pedido específico: `GET /api/pedidos/{id}`

Ejercicio 7

Indique si el siguiente uso es correcto o incorrecto y explique por qué:

```
GET https://api.ejemplo.com/eliminarUsuario/10
```

Solución: Es incorrecto. El método GET no debe usarse para eliminar recursos. La forma correcta sería usar: `DELETE /api/usuarios/10`

Ejercicio 8

Construya una petición HTTP completa (método, headers y body) para crear un nuevo usuario con nombre y correo.

Solución:

```
POST /api/usuarios HTTP/1.1
Host: api.ejemplo.com
Content-Type: application/json

{
  "nombre": "Juan Perez",
  "correo": "juan@email.com"
}
```

Ejercicio 9

Explique el rol de cada capa en una arquitectura web:

- Cliente
- API Gateway
- Backend
- Base de datos

Solución:

- Cliente: Interfaz donde el usuario interactúa (web o móvil).
- API Gateway: Punto de entrada que gestiona rutas, autenticación y tráfico.
- Backend: Lógica de negocio y procesamiento de datos.
- Base de datos: Almacenamiento persistente de información.

Ejercicio 10

Liste al menos 5 buenas prácticas al diseñar APIs REST y explique brevemente cada una.

Solución:

- Usar nombres claros en endpoints: Facilita la comprensión.
- Usar correctamente métodos HTTP: GET, POST, PUT, DELETE.
- Manejar códigos de estado: 200, 404, 500, etc.
- Versionar la API: Ej. /v1/, /v2/.
- Validar datos de entrada: Evita errores y vulnerabilidades.

Capítulo 2

Diseño de APIs

El diseño de una API es una decisión arquitectónica de largo plazo. Una API mal diseñada es difícil de usar, difícil de mantener y muy difícil de cambiar sin romper a los clientes que ya la consumen. Este capítulo presenta los principios y patrones que conducen a APIs claras, consistentes y durables.

2.1 Principios de Diseño REST

2.1.1 Recursos y Endpoints

En REST, todo es un **recurso**. Un recurso es cualquier entidad o concepto que pueda ser nombrado: un usuario, un pedido, un producto, una factura. La URL es la dirección única de ese recurso en el sistema.

Principio fundamental: Las URLs identifican *recursos* (sustantivos), no acciones (verbos). La acción la define el método HTTP.

```
# Incorrecto - verbos en la URL
GET  /obtenerUsuario/42
POST /crearUsuario
GET  /borrarUsuario?id=42

# Correcto - sustantivos, recursos
GET    /usuarios/42        # Obtener usuario 42
POST   /usuarios           # Crear usuario
DELETE /usuarios/42        # Eliminar usuario 42
```

Código 2.1. URLs correctas vs incorrectas en REST

2.1.2 Naming Conventions

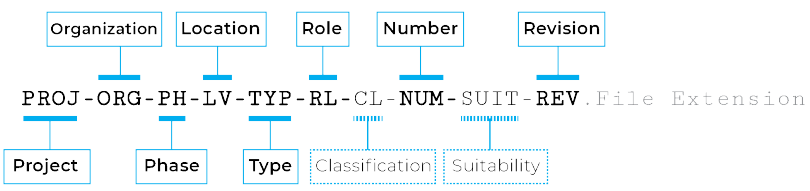


Figura 2.1. Naming Conventions - ISO19650

Una API predecible sigue convenciones de nomenclatura consistentes. Las siguientes reglas son el estándar de la industria:

- **Usa sustantivos en plural** para colecciones: `/usuarios`, `/productos`, `/pedidos`.
- **Usa minúsculas y guiones** para palabras compuestas: `/tipos-de-pago`, no `/TiposDePago` ni `/tipos_de_pago`.
- **Usa jerarquía de recursos** para relaciones padre-hijo: `/usuarios/42/pedidos` (pedidos del usuario 42).
- **Limita la profundidad** a máximo dos niveles de anidamiento para evitar URLs complejas.

2.1.3 Versionado de APIs

El versionado permite evolucionar una API sin romper a los clientes existentes. Existen tres estrategias principales:

Parámetros / Operandos

Estrategia 1 — Versionado en la URL (más común):

```
GET https://api.ejemplo.com/v1/usuarios
GET https://api.ejemplo.com/v2/usuarios
```

Estrategia 2 — Versionado en el Header:

```
GET https://api.ejemplo.com/usuarios
Accept: application/vnd.ejemplo.v2+json
```

Estrategia 3 — Versionado como query param:

```
GET https://api.ejemplo.com/usuarios?version=2
```

Nota Técnica

Recomendación: El versionado en la URL (`/v1/`, `/v2/`) es la estrategia más adoptada por su visibilidad y facilidad de uso. Es el enfoque utilizado por Stripe, GitHub, Twitter y la mayoría de las grandes APIs públicas. Mantén siempre al menos la versión anterior activa durante un período de transición (generalmente 6 a 12 meses).

2.1.4 Buenas Prácticas REST

- **Statelessness:** Cada petición debe contener toda la información necesaria. El servidor no debe almacenar el estado del cliente entre peticiones.
- **Paginación:** Para colecciones grandes, utiliza parámetros de paginación: `?page=2&limit=20` o cursores para mayor eficiencia.
- **Filtrado y ordenación:** Soporta `?rol=editor&sort=nombre` para consultas flexibles sin multiplicar los endpoints.
- **Respuestas consistentes:** Usa siempre la misma estructura de respuesta, incluyendo metadatos en colecciones (`total`, `page`, `hasMore`).

- **Manejo de errores descriptivo:** Devuelve mensajes de error claros con código de estado apropiado y detalle del problema.

2.2 Diseño Orientado a Recursos

2.2.1 CRUD y Operaciones Estándar

El patrón **CRUD** (Create, Read, Update, Delete) es la base del diseño REST. Para cada recurso se definen operaciones estándar mapeadas a métodos HTTP:

Parámetros / Operandos

Operación	Método	Endpoint	Descripción
Create	POST	`/usuarios`	Crea un nuevo usuario
Read (lista)	GET	`/usuarios`	Lista todos los usuarios
Read (uno)	GET	`/usuarios/{id}`	Obtiene usuario por ID
Update (completo)	PUT	`/usuarios/{id}`	Reemplaza el recurso
Update (parcial)	PATCH	`/usuarios/{id}`	Actualiza campos específicos
Delete	DELETE	`/usuarios/{id}`	Elimina el recurso

2.2.2 Relaciones entre Entidades

Los recursos raramente existen de forma aislada. Las relaciones entre entidades se modelan mediante **rutas anidadas** o **referencias por ID**:

```
# Relacion 1:N - Pedidos de un usuario
GET  /usuarios/42/pedidos          # Todos los pedidos del
   usuario 42
GET  /usuarios/42/pedidos/101      # Pedido 101 del usuario 42
POST /usuarios/42/pedidos          # Crear pedido para usuario 42

# Relacion N:M - Productos en un pedido
GET  /pedidos/101/productos        # Productos del pedido 101
POST /pedidos/101/productos/55     # Agregar producto 55 al
   pedido 101
```

Código 2.2. Modelado de relaciones en URLs REST

2.2.3 Idempotencia

La **idempotencia** es una propiedad crítica: una operación idempotente produce el mismo resultado sin importar cuántas veces se ejecute. Este concepto es fundamental para diseñar APIs resilientes ante fallos de red.

Parámetros / Operandos

Método	¿Idempotente?	Razonamiento
GET	Sí	Leer no modifica el estado del servidor.
PUT	Sí	Reemplazar con los mismos datos produce el mismo recurso.
DELETE	Sí	Borrar un recurso ya borrado da 404, el estado es el mismo.
POST	No	Crear dos veces produce dos recursos distintos.
PATCH	No*	Depende de la implementación (ej: incrementos).

Advertencia

Idempotencia en clientes: Cuando un cliente no recibe respuesta (timeout), debe poder reintentar de forma segura. Si usas POST y no garantizas idempotencia, implementa un **Idempotency-Key** en el header para que el servidor detecte y devuelva la misma respuesta sin procesar dos veces la operación (patrón usado por Stripe y Adyen).

2.3 Documentación

2.3.1 Uso de Swagger / OpenAPI

OpenAPI (antes Swagger) es el estándar de la industria para describir APIs RESTful. Una especificación OpenAPI es un archivo YAML o JSON que define de forma exhaustiva todos los endpoints, parámetros, esquemas de datos, códigos de respuesta y requisitos de autenticación de una API.

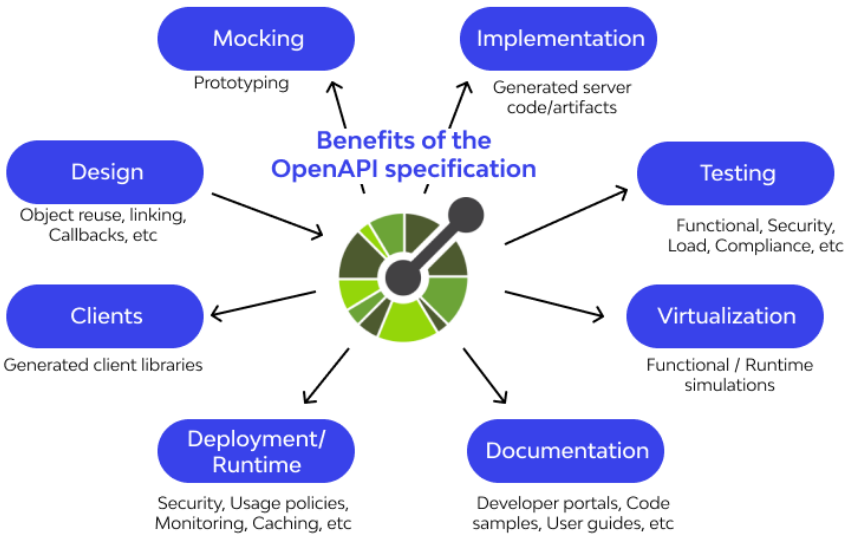


Figura 2.2. Caracterìsticas OpenAPI

```yaml
openapi: 3.0.3
info:
  title: API de Usuarios
  version: "1.0"
  description: Gestión completa de usuarios del sistema

paths:
  /usuarios/{id}:
    get:
      summary: Obtener usuario por ID
      parameters:
        - name: id
          in: path
          required: true
          schema:
            type: integer
      responses:
        "200":
          description: Usuario encontrado
          content:
            application/json:
              schema:
                $ref: "#/components/schemas/Usuario"
        "404":
          description: Usuario no encontrado

components:
  schemas:
    Usuario:
      type: object
      properties:
        id:
          type: integer
        nombre:
          type: string
        email:
          type: string
          format: email
```

Código 2.3. Fragmento de especificación OpenAPI 3.0

2.3.2 Generación Automática de Documentación

La especificación OpenAPI permite generar documentación interactiva automáticamente. Las herramientas más populares son:

- **Swagger UI:** Interfaz web que renderiza la especificación y permite ejecutar peticiones directamente desde el navegador. Se integra en frameworks como FastAPI (Python), Spring Boot (Java) y NestJS (Node.js).
- **Redoc:** Documentación visualmente superior, ideal para APIs públicas. Genera páginas estáticas a partir del archivo OpenAPI.

- **Stoplight:** Plataforma colaborativa para diseño y documentación de APIs con control de versiones integrado.

```python
from fastapi import FastAPI

app = FastAPI(
    title="API de Usuarios",
    description="Gestión de usuarios - Colección Bosón",
    version="1.0.0",
    docs_url="/docs",        # Swagger UI en /docs
    redoc_url="/redoc"       # Redoc en /redoc
)

@app.get("/usuarios/{id}", tags=["usuarios"])
async def obtener_usuario(id: int):
    """Retorna el usuario identificado por su ID."""
    return {"id": id, "nombre": "Ana Quispe"}
```

Código 2.4. Integración de Swagger UI en FastAPI (Python)

2.3.3 Testing Interactivo

Una de las mayores ventajas de Swagger UI es la posibilidad de realizar **testing interactivo** directamente desde la documentación. Esto permite:

1. Explorar todos los endpoints disponibles sin escribir código.
2. Enviar peticiones reales con parámetros personalizados.
3. Inspeccionar las respuestas, headers y códigos de estado.
4. Probar la autenticación introduciendo tokens directamente en la interfaz.
5. Compartir la URL de documentación con el equipo de frontend para reducir la fricción en la integración.

Ejemplo Analítico — Paso a Paso

Flujo de testing interactivo con Swagger UI:

1. Navega a `http://localhost:8000/docs` en tu navegador.
2. Haz clic en *Authorize* e ingresa tu Bearer token.
3. Expande el endpoint `GET /usuarios/{id}`.
4. Haz clic en *Try it out* e ingresa un ID válido.
5. Pulsa *Execute* y observa la respuesta en tiempo real.
6. Revisa el código de estado, headers y el payload JSON retornado.

Nota Técnica

Contract-First vs Code-First: Existen dos enfoques para trabajar con OpenAPI. En el enfoque *Contract-First*, el equipo diseña la especificación YAML antes de escribir código, lo que facilita el trabajo en paralelo entre frontend y backend.

En el enfoque *Code-First*, la especificación se genera automáticamente a partir de anotaciones en el código (como hace FastAPI). Para APIs públicas o proyectos con múltiples equipos, se recomienda el enfoque *Contract-First*.

2.4 Ejercicios Resueltos — Implementación paso a paso

Ejercicio 1

En la ciudad de La Paz, una institución necesita registrar ciudadanos mediante consola.

Solución:

1. Se solicita nombre y edad al usuario
2. Se almacenan los datos en variables
3. Se muestra la información formateada

```
nombre = input("Nombre: ")
edad = int(input("Edad: "))

print(f"Ciudadano registrado: {nombre}, {edad} años")
```

Código 2.5. Registro de ciudadano

Ejercicio 2

En Santa Cruz, una tienda necesita actualizar precios de productos.

Solución:

1. Se define un producto con precio
2. Se solicita nuevo precio
3. Se actualiza el valor

```
producto = "Laptop"
precio = 3500

nuevo_precio = float(input("Nuevo precio: "))
precio = nuevo_precio

print(f"Producto: {producto} | Precio actualizado: {precio}")
```

Código 2.6. Actualización de producto

Ejercicio 3

En Cochabamba, un banco necesita consultar transacciones.

Solución:

1. Se simula una lista de transacciones

2. Se muestran los datos al usuario

```
transacciones = [150, -200, 300, -50]

print("Historial de transacciones:")
for t in transacciones:
    print(f"Monto: {t}")
```

Código 2.7. Consulta de transacciones

Ejercicio 4

En El Alto, un hospital necesita filtrar pacientes por edad.
Solución:

1. Se define una lista de edades
2. Se solicita edad mínima
3. Se filtran los datos

```
edades = [25, 40, 60, 30, 55]

edad_min = int(input("Edad mínima: "))

for e in edades:
    if e >= edad_min:
        print(f"Paciente válido: {e} años")
```

Código 2.8. Filtro de pacientes

Ejercicio 5

En La Paz, una empresa logística necesita validar compras con stock.
Solución:

1. Se define el stock inicial
2. Se solicita cantidad a comprar
3. Se valida disponibilidad
4. Se actualiza stock si es válido

```
producto = "Laptop"
stock = 10

cantidad = int(input("Cantidad a comprar: "))

if cantidad <= stock:
    stock -= cantidad
    print(f"Compra realizada. Stock restante: {stock}")
else:
    print("Error: Stock insuficiente")
```

Código 2.9. Control de stock

Ejercicio 6

En la ciudad de La Paz, una empresa de delivery necesita validar pedidos considerando múltiples condiciones: existencia del producto, stock disponible y cálculo del total con impuesto.

Solución (paso a paso):

1. Se define un catálogo de productos con precio y stock
2. Se solicita el producto y cantidad
3. Se valida existencia del producto
4. Se valida stock disponible
5. Se calcula el total con impuesto (13%)
6. Se actualiza el stock

```
productos = {
    "laptop": {"precio": 5000, "stock": 5},
    "mouse": {"precio": 100, "stock": 20}
}

nombre = input("Producto: ").lower()
cantidad = int(input("Cantidad: "))

if nombre in productos:
    producto = productos[nombre]

    if cantidad <= producto["stock"]:
        subtotal = producto["precio"] * cantidad
        total = subtotal * 1.13

        producto["stock"] -= cantidad

        print(f"Compra exitosa")
        print(f"Total con impuesto: {total:.2f}")
        print(f"Stock restante: {producto['stock']}")
    else:
        print("Stock insuficiente")
else:
    print("Producto no existe")
```

Código 2.10. Sistema de pedidos con validación completa

Ejercicio 7

En Santa Cruz, una empresa financiera necesita registrar múltiples usuarios y guardar los datos en un archivo para auditoría.

Solución:

1. Se solicita cantidad de usuarios a registrar
2. Se capturan datos en un ciclo
3. Se almacenan en una lista

4. Se guardan en un archivo
5. Se leen nuevamente para verificación

```python
usuarios = []

n = int(input("Cantidad de usuarios: "))

for i in range(n):
    nombre = input("Nombre: ")
    edad = int(input("Edad: "))

    usuarios.append(f"{nombre},{edad}")

# Guardar en archivo
with open("usuarios.txt", "w", encoding="utf-8") as f:
    for u in usuarios:
        f.write(u + "\n")

# Leer archivo
print("\nUsuarios registrados:")
with open("usuarios.txt", "r", encoding="utf-8") as f:
    for linea in f:
        print(linea.strip())
```

Código 2.11. Registro masivo con persistencia en archivo

Ejercicio 8

En la ciudad de La Paz, una empresa de telecomunicaciones analiza el consumo mensual (en GB) de sus usuarios para identificar patrones.

Solución:

1. Se define una lista extensa de consumos
2. Se calculan métricas estadísticas: media, máximo y mínimo
3. Se muestran resultados formateados

```python
consumos = [5, 10, 15, 20, 8, 12, 25, 30, 18, 22, 7, 9, 14, 16,
   28]

media = sum(consumos) / len(consumos)
maximo = max(consumos)
minimo = min(consumos)

print(f"{'Métrica':<12} {'Valor':>8}")
print(f"{'Media':<12} {media:>8.2f}")
print(f"{'Máximo':<12} {maximo:>8}")
print(f"{'Mínimo':<12} {minimo:>8}")
```

Código 2.12. Análisis de consumo de datos

Ejercicio 9

En Santa Cruz, una empresa de transporte analiza tiempos de entrega (en minutos) para evaluar eficiencia operativa.

Solución:

1. Se define un conjunto amplio de tiempos
2. Se calcula media, varianza y desviación estándar
3. Se interpretan resultados

```python
import math

tiempos = [30, 45, 50, 60, 35, 40, 55, 65, 70, 48, 52, 38, 42,
   58, 62]

media = sum(tiempos) / len(tiempos)

varianza = sum((x - media)**2 for x in tiempos) / len(tiempos)
desviacion = math.sqrt(varianza)

print(f"Media: {media:.2f}")
print(f"Varianza: {varianza:.2f}")
print(f"Desviación estándar: {desviacion:.2f}")
```

Código 2.13. Análisis estadístico de tiempos

Ejercicio 10

En Cochabamba, una empresa financiera desea aplicar ciencia de datos para analizar ingresos de clientes y segmentarlos.

Solución:

1. Se carga un conjunto de datos
2. Se utiliza una librería de análisis (pandas)
3. Se calculan estadísticas descriptivas
4. Se clasifican clientes en categorías

```python
import pandas as pd

# Datos simulados
data = {
    "cliente": ["A","B","C","D","E","F","G","H","I","J"],
    "ingreso": [1200, 2500, 1800, 3000, 2200, 2700, 3500, 1600,
       2900, 3100]
}

df = pd.DataFrame(data)

# Estadísticas
print(df.describe())
```

```
13
14 # Segmentación simple
15 def clasificar(ingreso):
16     if ingreso < 2000:
17         return "Bajo"
18     elif ingreso < 3000:
19         return "Medio"
20     else:
21         return "Alto"
22
23 df["segmento"] = df["ingreso"].apply(clasificar)
24
25 print("\nSegmentación:")
26 print(df)
```

Código 2.14. Análisis con ciencia de datos

2.5 Ejercicios Propuestos

Ejercicio 1

En la ciudad de La Paz, una empresa municipal necesita exponer datos de sensores ambientales. Diseñe endpoints REST para:

- Registrar datos de temperatura
- Consultar datos históricos
- Filtrar datos por fecha

Ejercicio 2

Una empresa privada de monitoreo industrial requiere autenticar dispositivos IoT. Proponga un diseño de API que permita:

- Registro de dispositivos
- Autenticación mediante token
- Envío seguro de datos

Ejercicio 3

En Santa Cruz, una empresa logística desea monitorear ubicación de vehículos en tiempo real. Diseñe endpoints que permitan:

- Enviar coordenadas GPS
- Consultar última ubicación
- Obtener historial de rutas

Ejercicio 4

Una institución educativa desea construir una API para gestionar estudiantes y cursos. Defina:

- Estructura de endpoints
- Relaciones entre entidades
- Métodos HTTP utilizados

Ejercicio 5

Una empresa financiera requiere controlar accesos a su API. Diseñe un sistema que incluya:

- Uso de API Keys
- Limitación de peticiones (rate limiting)
- Registro de logs de acceso

Ejercicio 6

En Cochabamba, una empresa agrícola utiliza sensores para medir humedad del suelo. Diseñe una API que permita:

- Registrar datos periódicos
- Consultar datos por zona
- Generar reportes básicos

Ejercicio 7

Una empresa de salud necesita manejar historiales clínicos mediante API. Plantee:

- Endpoints principales
- Seguridad de datos sensibles
- Control de acceso por roles

Ejercicio 8

Una startup tecnológica desea integrar múltiples microservicios. Diseñe una arquitectura basada en APIs considerando:

- Comunicación entre servicios
- Uso de API Gateway
- Manejo de errores y resiliencia

Ejercicio 9

En las ciudades de La Paz y El Alto, una institución académica y de investigación desea implementar un sistema distribuido de monitoreo ambiental utilizando al menos 6 dispositivos ESP32 ubicados estratégicamente en diferentes zonas geográficas (urbana, periurbana y rural) y a distintas alturas (por ejemplo: 3200, 3500, 3800 y 4000 msnm). El propósito es analizar el comportamiento de variables como la temperatura y la humedad relativa en función del tiempo, la ubicación y la altitud, con fines científicos y de investigación aplicada.

Cada dispositivo ESP32 debe cumplir con los siguientes requerimientos: medir temperatura (en °C) y humedad (en %), registrar la fecha y hora exacta de cada medición, enviar los datos automáticamente cada 5 segundos hacia un servidor central, y operar únicamente en el rango horario de 08:00 a 12:00 durante 2 días consecutivos. Además, cada dispositivo debe contar con un identificador único y enviar información adicional como zona, altitud y coordenadas GPS (latitud y longitud).

Se requiere diseñar un sistema completo que contemple los siguientes aspectos:

- Definir la arquitectura general del sistema, incluyendo cómo los dispositivos enviarán los datos (por ejemplo, mediante HTTP o MQTT), qué tipo de backend se utilizará (como FastAPI o Node.js) y cómo será el flujo completo de información desde los sensores hasta el almacenamiento.
- Diseñar una API REST que permita recibir y consultar los datos generados. Se deben proponer endpoints claros para el registro de datos, consulta por dispositivo, filtrado por fecha, zona o altitud, y generación de resúmenes.
- Especificar qué información debe enviarse en cada petición. Por ejemplo, cada registro debería incluir: identificador del dispositivo, temperatura, humedad, latitud, longitud, altitud, zona y timestamp.
- Diseñar un modelo de base de datos adecuado para almacenar grandes volúmenes de información generados cada 5 segundos por múltiples dispositivos durante varios días, considerando eficiencia y escalabilidad.
- Proponer un análisis estadístico de los datos recolectados, incluyendo el cálculo de métricas como media, varianza, desviación estándar, así como comparaciones entre diferentes zonas y alturas.
- Plantear un enfoque de ciencia de datos que permita construir un modelo predictivo (por ejemplo, regresión lineal u otro algoritmo de machine learning) para estimar el comportamiento futuro de la temperatura o la humedad en función de variables como altitud, ubicación y hora del día.
- Diseñar endpoints adicionales en la API que permitan consultar los resultados del análisis y las predicciones generadas, facilitando su uso en investigaciones posteriores.
- Finalmente, justificar las decisiones de diseño considerando aspectos como escalabilidad del sistema, seguridad de la API, eficiencia en el manejo de datos y su aplicabilidad en contextos reales de investigación científica.

Ejercicio 10

En un centro de salud de la ciudad de La Paz, se desea implementar una API sencilla para registrar y consultar imágenes médicas básicas (por ejemplo, fotografías de heridas o radiografías simples) de pacientes, con el objetivo de llevar un control digital de los estudios realizados.

Cada registro debe contener información mínima como:

- ID del paciente
- Nombre del paciente
- Tipo de estudio
- Imagen (archivo o en formato base64)
- Fecha del registro

Se pide diseñar una API REST simple que permita:

- Subir una imagen médica al sistema
- Consultar todas las imágenes registradas
- Consultar imágenes de un paciente específico

Además, se solicita:

- Proponer endpoints básicos para estas operaciones
- Definir qué datos se deben enviar en cada petición
- Plantear una estructura sencilla de almacenamiento (base de datos o archivos)

Bibliografía

[1] Roy Thomas Fielding, *Architectural Styles and the Design of Network-based Software Architectures*, Tesis Doctoral, University of California, Irvine, 2000.

[2] Mark Massé, *REST API Design Rulebook*, O'Reilly Media, 2011.

[3] Leonard Richardson y Mike Amundsen, *RESTful Web APIs*, O'Reilly Media, 2013.

[4] Jim Webber, Savas Parastatidis y Ian Robinson, *REST in Practice*, O'Reilly Media, 2010.

[5] OpenAPI Initiative, *OpenAPI Specification 3.1.0*, `https://spec.openapis.org/oas/v3.1.0`, 2021.

[6] Sam Newman, *Building Microservices*, O'Reilly Media, 2015.

Configuración del Entorno de Desarrollo

.1 Instalación de FastAPI y Uvicorn

1. Crear el entorno virtual:

```
conda create -n apis_web python=3.11
conda activate apis_web
pip install fastapi uvicorn[standard] httpx pytest
```

2. Crear el archivo `main.py` con el servidor básico:

```
from fastapi import FastAPI
app = FastAPI()

@app.get("/")
def raiz():
    return {"mensaje": "API activa"}
```

3. Levantar el servidor de desarrollo:

```
uvicorn main:app --reload --port 8000
```

4. Abrir `http://localhost:8000/docs` para acceder a Swagger UI.

.2 Herramientas de Testing de APIs

1. **curl** — Cliente HTTP en línea de comandos (preinstalado en Linux/macOS):

```
curl -X GET "http://localhost:8000/usuarios/42" \
     -H "Authorization: Bearer TOKEN" \
     -H "Accept: application/json"
```

2. **Postman** — Descargable desde `https://www.postman.com`. Interfaz gráfica para construcción, ejecución y documentación de colecciones de pruebas.
3. **HTTPie** — Alternativa moderna a curl con sintaxis amigable:

```
pip install httpie
http GET localhost:8000/usuarios/42 Authorization:"Bearer
   TOKEN"
```

Recursos y Referencias en Línea

- **OpenAPI Specification** — https://spec.openapis.org: especificación oficial.
- **Swagger Editor** — https://editor.swagger.io: editor online de especificaciones.
- **FastAPI Docs** — https://fastapi.tiangolo.com: framework Python con OpenAPI integrado.
- **REST API Guidelines** — https://github.com/microsoft/api-guidelines: guías de Microsoft.
- **Postman Learning Center** — https://learning.postman.com: tutoriales interactivos.
- **HTTP Status Codes** — https://httpstatuses.com: referencia completa de códigos.
- **JSONPlaceholder** — https://jsonplaceholder.typicode.com: API pública de prueba.